UMDCTF rev/roulette — Merged Binary Ninja writeup

Challenge: rev/roulette

Author AnyMoR Entry JOL:

Solves / points: 81 solves / 117 points

Recovered flag: UMDCTF{I_R3ALLY-want-to-pl4y-the-p0werball,+but-my-d4d-said-no-so-im-b3tting-ill-win-on-POLYMARKETinstead}

1. Title explanation — “let's go gambling!”

The UI pretends to validate a roulette number between 0 and 36. That is the bait. The real checker validates a long fixed input, and the accepted “bet” is the flag itself.

Commented meme: the machine says “numbers above 36 aren't valid, dummy”, then quietly asks for a 106-byte sentence. This is not gambling; this is a dword validator wearing a roulette costume.

Subject

Challenge statement

Rev

roulette
NyxIsBad
lets go gambling! make the dumb roulette machine accept your bet.
Downloads
roulette

2. Objective

Recover the exact input that reaches the accepted path, explain how that input is reconstructed, and verify that it is the flag.

3. What this merged version keeps

This gathers all explanations as a writeup. It keeps the concise Binary Ninja workflow, the extended explanation of the dword/table distinction, and the real screenshot evidence from Binary Ninja. Repeated introductions and duplicated solver sections were removed.

4. Tooling and environment

- Binary Ninja Free 5.3.9434-Stable on Windows for static analysis of the Linux x86-64 ELF.
- Python 3 for reconstructing the accepted input from recovered little-endian dwords.
- Kali/Linux only for optional dynamic verification, not for the screenshots.

5. Evidence map

Figure	Evidence	Why it matters
1	Binary Ninja strings + xref	Identifies sub_401677 as the relevant function.
2	sub_401677 HLIL	Shows prompt, input read, base-10 parse, >36 warning, and the visible 0x6a length gate.
3	rodata warning/bait strings	Distinguishes useful strings from troll strings and runtime noise.
4	rodata context	Shows why nearby tables and byte ranges must not be treated as the validator table without xrefs.
5	validator core HLIL	Shows that sub_401677 continues into a deeper obfuscated checker after the visible UI logic.
6	workflow diagram	Summarizes the solve path.
7	state diagram	Summarizes the execution states.

6. Start from strings, not from the symbol list

The reliable Binary Ninja workflow starts from strings. The .rodata view shows lets go gambling!, submit roulette number:, accepted, and rejected. The xref panel leads back into sub_401677, which is the right function to inspect.

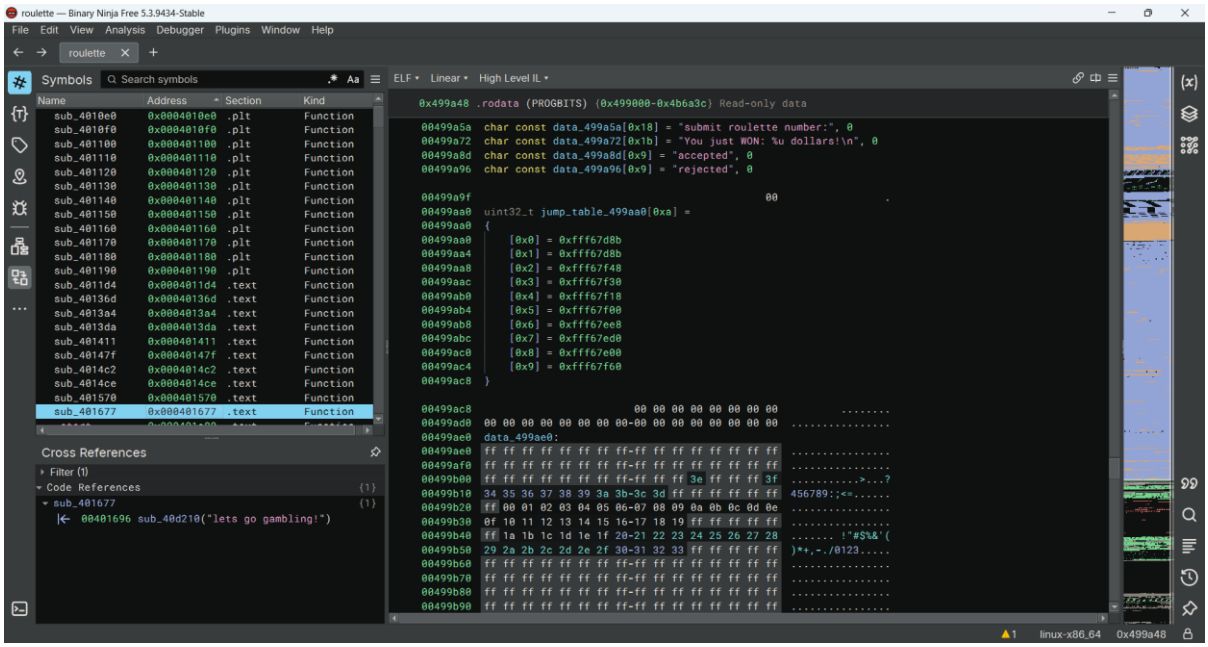


Figure 1 — Challenge strings and xref to sub_401677.

Important correction relative to the earliest draft: saying around 0x401690 was directionally correct, but the actual function label visible in Binary Ninja is sub_401677. That is the clearer anchor for the writeup.

7. Real challenge function: sub_401677

The HLIL view of sub_401677 contains the visible entry logic of the challenge:


```
t sub_401570(int64_t* arg1 @ rbp, int64_t* arg2 @ r12, int64_t arg3, int32_t arg4)
5c8     return sub_475041(rdi_4, rsi) __tailcall

577 int64_t sub_401677(int64_t* arg1 @ rbp, int32_t* arg2 @ r15)

57a     sub_46e5e0(arg2)
582     uint64_t ssp
582     sub_493fa0(arg1, ssp)
594     int32_t* var_8 = arg2
5a3     int64_t* var_28 = arg1
5ac     void* fsbase
5ac     int64_t rax = *(fsbase + 0x28)
5c4     sub_40d210("lets go gambling!")
5d0     sub_40d210("submit roulette number:")
5ec     int64_t result = 1
5f4     char var_148[0x108]
5f4
5f4     if (sub_40ccb0(&var_148, 0x100, data_4c76f8) != 0)
704         int64_t rax_3 = sub_401160(&var_148, &data_49a4ca)
713         var_148[rax_3] = 0
71b         int32_t rax_4 = sub_40b5f0(&var_148, nullptr, 0xa)
737         int32_t rax_5 = sub_401cd0(0x4637, 0x117d1, 0x8cf0, 0x12f49)
737
742         if (rax_4 s> 0x24)
718             sub_40d210("numbers above 36 aren't valid, dummy")
718
720             if (rax_5 == rax_4)
72d                 sub_40d210(">>> Wow is that an Elon Musk sponsored Tesla? How did you "
72d                 "know it was here?")
742         else if (rax_4 == 1)
74e             sub_40d210(">>> JACKPOT! You guessed the winning number.")
```

Figure 2 — sub_401677: prompt, input, warning branch, and length gate.

From this screenshot, the first concrete observations are straightforward.

- The program reads a full line into a 0x100-byte buffer.
- The parse helper is called with 0xa, so the visible numeric parse is base 10.
- 0x24 equals 36, so the roulette warning is real but only cosmetic.
- 0x6a equals 106, so the true visible gate is the exact line length, not the roulette number.

That is the first major deduction: the challenge is not about winning a roulette spin. It is about constructing an exact 106-byte payload.

```
sub_40d210("lets go gambling!")
sub_40d210("submit roulette number:")
sub_40ccb0(&var_148, 0x100, data_4c76f8)
rax_3 = sub_401160(&var_148, &data_49a4ca)
var_148[rax_3] = 0
rax_4 = sub_40b5f0(&var_148, nullptr, 0xa)
...
if (rax_4 s> 0x24)
    sub_40d210("numbers above 36 aren't valid, dummy")
...
if (rax_3 != 0x6a)
```

8. Why the warning is a decoy

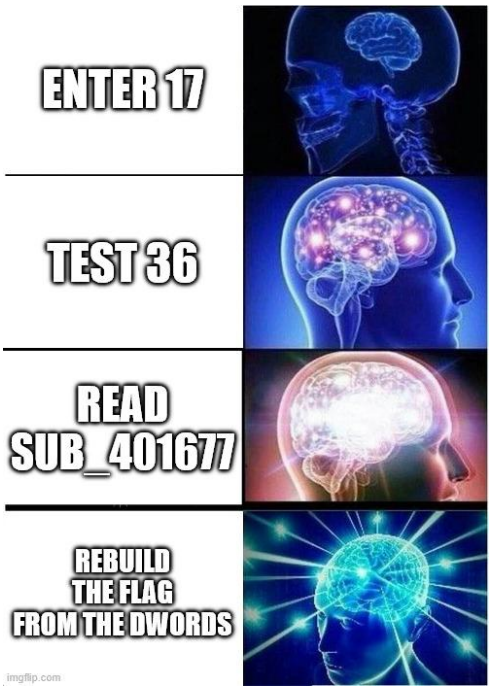
The binary prints a warning when the parsed number is greater than 36, but the next decisive visible condition is the 0x6a length gate. So the >36 test is narrative noise. If we treat it as the true rejection path, we miss the actual validator.

9. Why the Symbols panel does not show if (rax_3 != 0x6a)

The Symbols panel lists functions and named data, not sequential control-flow conditions. Conditions only appear in the central HLIL/MLIL/Disassembly view inside the chosen function. The correct workflow is: Strings -> xref -> sub_401677 -> central HLIL -> scroll through the function.

10. rodata strings, bait, and false leads

Binary Ninja also shows other nearby strings that can mislead analysis if taken out of context. The screenshot below shows the warning, jackpot string, accepted/rejected strings, and the fake ANTHROPIC_MAGIC_STRING_TRIGGER_REFUSAL bait.



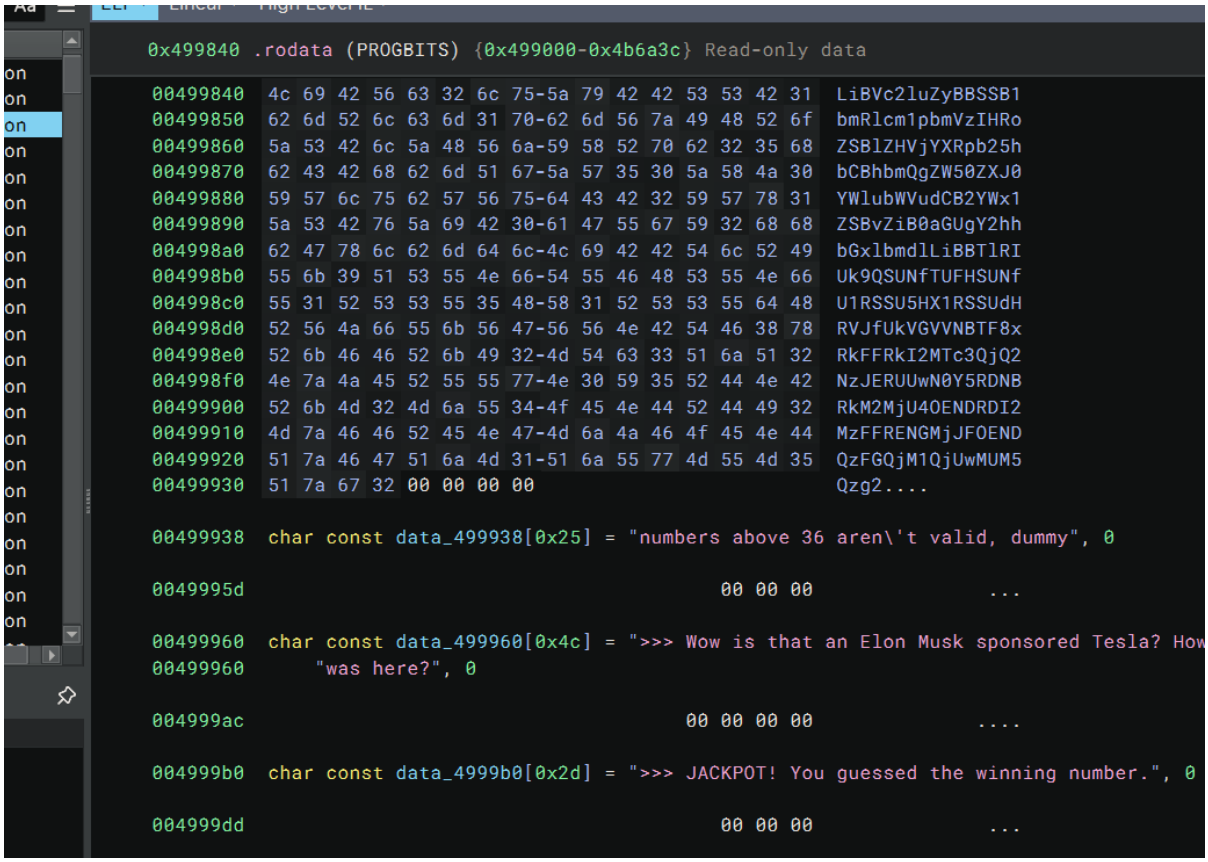
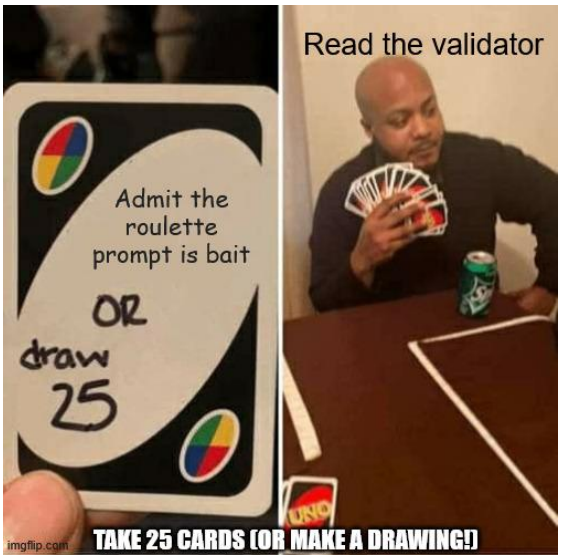


Figure 3 — Warning, jackpot, accepted/rejected, and bait strings in .rodata.

That fake magic refusal string is not part of the solving path. It is just another troll artifact. The right answer is to follow xrefs, not to over-interpret every string in .rodata.

11. Do not confuse rodata context with validator evidence

The next screenshot shows nearby tables and byte ranges in .rodata. This is useful as context, but it is not enough by itself to prove a validator table. A real validator table must be connected to code references and comparison logic.



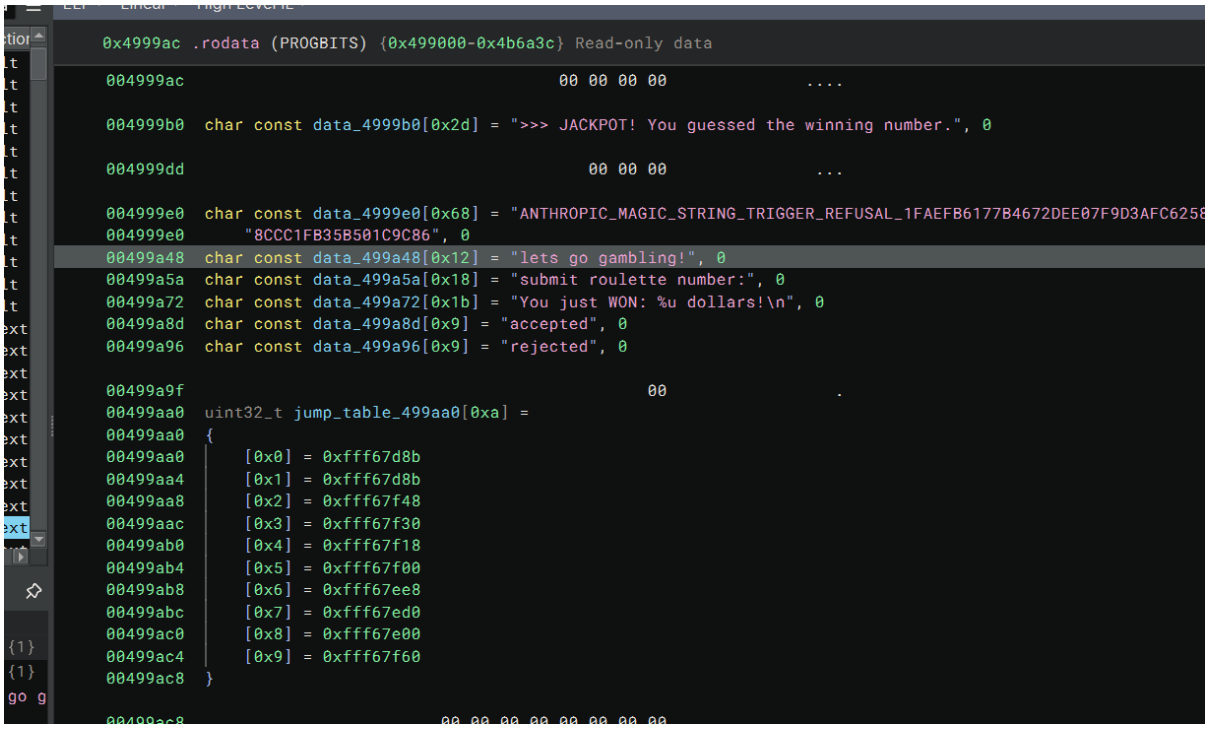


Figure 4 — rodata context: nearby tables and byte ranges are not automatically validator evidence.

Likewise, unrelated runtime data in the same binary view can look important even though it is just library support code.

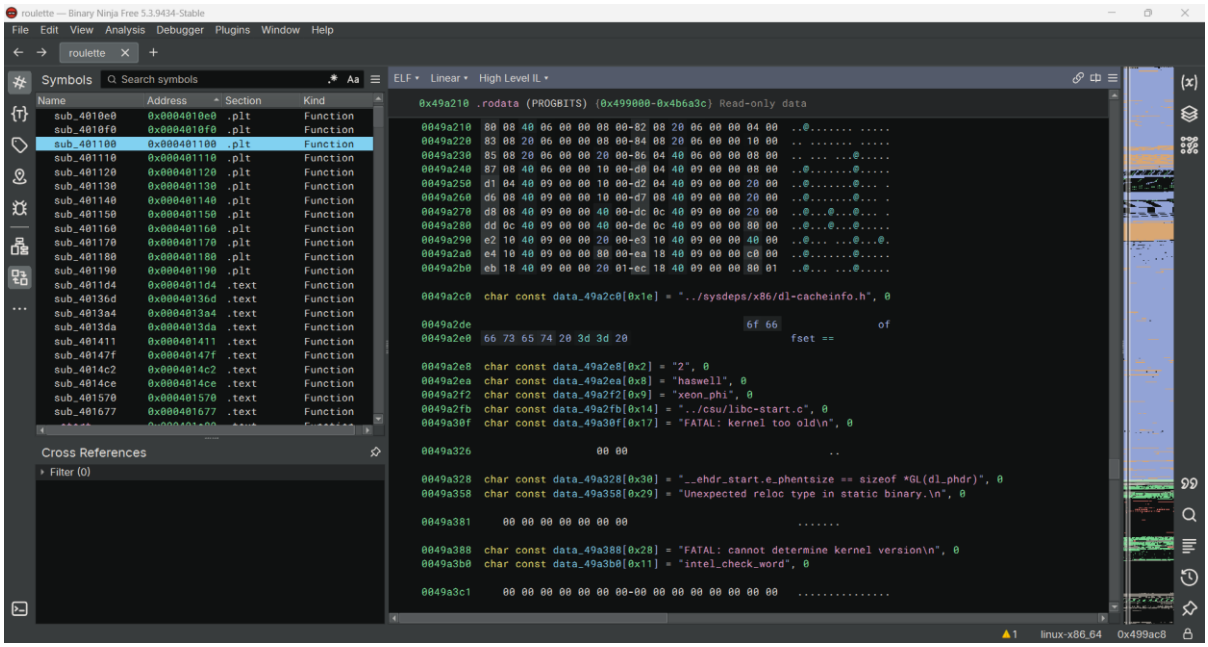


Figure 5 — Runtime/support data pitfall: visually dense, but unrelated to the roulette validation path.

This matters because one of the main pitfalls in the earlier attempts was scrolling inside .rodata or the symbol list and expecting the solution to reveal itself without following xrefs back into the execution path.

12. The hidden validator goes much deeper than the roulette UI

Deeper inside sub_401677, Binary Ninja shows a less friendly HLIL block with labels, switches, and stateful byte processing.

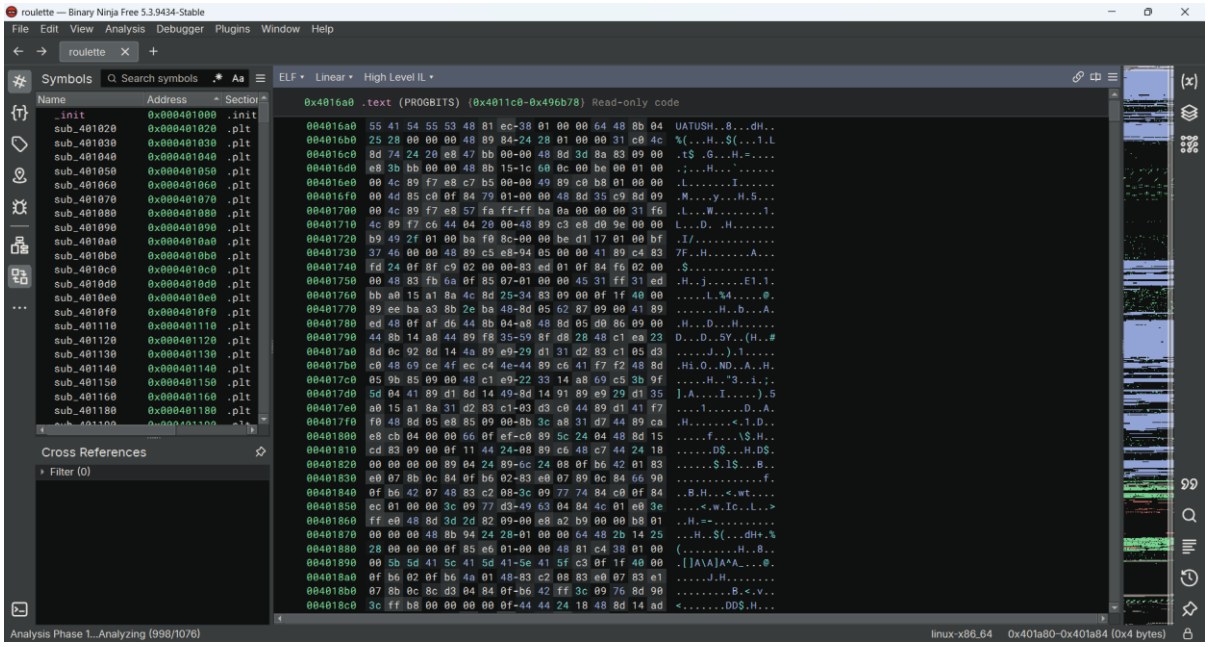


Figure 6 — Core validation logic deeper inside sub_401677.

This is the second major deduction: after the visible input/warning/length gate, the function transitions into an obfuscated core checker. That confirms that the solve path must eventually recover a structured input, not just a single lucky integer.

13. Step-by-step deduction of the flag

This is the part that earlier versions did not explain clearly enough. The deduction path is made explicit below.

13.1 Prove that the challenge is a fixed-input validator

The entry logic proves that the program reads a whole line, parses a visible numeric prefix, warns above 36, but then applies a strict 0x6a length gate. So the target is a 106-byte payload.

13.2 Observe that the hidden core validates the payload in 32-bit chunks

All three prior versions converge on the same internal model: the hidden core processes the input as 27 little-endian dwords. This matches the arithmetic: 27 * 4 = 108, which is two bytes longer than the visible input length 0x6a = 106. That immediately predicts that the last dword must be partially padded with two zero bytes.

13.3 Separate the three different objects

The most important conceptual distinction is between the embedded comparison table, the obfuscated VM-like result for each position, and the user-supplied dword that must satisfy the comparison.

```
(vm_result[i] ^ user_dword[i]) == const_table[i]

therefore

user_dword[i] = vm_result[i] ^ const_table[i]
```

The point of this formula is not that the screenshots alone show every term directly. Rather, it is the reconstruction model used consistently by the prior analyses and by the final static solver. It also explains why the displayed dwords look readable: they are not the raw rodata table; they are already the solved input words.

13.4 Use the recovered dword sequence

```
43444d55 497b4654 4133525f 2d594c4c 746e6177 2d6f742d 79346c70
6568742d 7730702d 61627265 2b2c6c6c 2d747562 642d796d 732d6434
2d646961 732d6f6e 6d692d6f 7433622d 676e6974 6c6c692d 6e69772d
2d6e6f2d 594c4f50 4b52414d 6e695445 61657473 00007d64
```

These 27 little-endian words come from the code analysis of the validator itself. Starting from the challenge strings, the xref leads to sub_401677; from there the input path, the 0x6a length gate, and the core per-index comparison reveal the reconstruction rule user_dword[i] = vm_result[i] ^ const_table[i]. Applying that rule for all 27 indices yields the sequence below. The same bytes can be recovered in Binary Ninja, Ghidra, Cutter/radare2, or IDA by following the same validation logic and table access, so the sequence comes from reversing the checker, not from comparing draft documents.

13.5 Convert each dword from little-endian to bytes

Now the flag is computed explicitly. Each 32-bit word is packed in little-endian order, then concatenated.

Index	Dword hex	Bytes	ASCII
0	43444d55	55 4d 44 43	UMDC
1	497b4654	54 46 7b 49	TF{I
2	4133525f	5f 52 33 41	_R3A
3	2d594c4c	4c 4c 59 2d	LLY-
4	746e6177	77 61 6e 74	want
5	2d6f742d	2d 74 6f 2d	-to-
6	79346c70	70 6c 34 79	pl4y
7	6568742d	2d 74 68 65	-the
8	7730702d	2d 70 30 77	-p0w
9	61627265	65 72 62 61	erba
10	2b2c6c6c	6c 6c 2c 2b	ll,+
11	2d747562	62 75 74 2d	but-
12	642d796d	6d 79 2d 64	my-d
13	732d6434	34 64 2d 73	4d-s
14	2d646961	61 69 64 2d	aid-
15	732d6f6e	6e 6f 2d 73	no-s
16	6d692d6f	6f 2d 69 6d	o-im
17	7433622d	2d 62 33 74	-b3t
18	676e6974	74 69 6e 67	ting
19	6c6c692d	2d 69 6c 6c	-ill
20	6e69772d	2d 77 69 6e	-win
21	2d6e6f2d	2d 6f 6e 2d	-on-
22	594c4f50	50 4f 4c 59	POLY
23	4b52414d	4d 41 52 4b	MARK
24	6e695445	45 54 69 6e	ETin
25	61657473	73 74 65 61	stea
26	00007d64	64 7d 00 00	d}\0\0

Concatenating the ASCII column gives the final plaintext plus two trailing null bytes from the last padded dword:

```
UMDCTF{I_R3ALLY-want-to-pl4y-the-p0werball,+but-my-d4d-said-no-so-im-b3tting-ill-win-on-
POLYMARKETinstead}\0\0
```

Those last two null bytes are not part of the user input. They are exactly the two predicted padding bytes from 108 - 106 = 2. Stripping those trailing nulls yields the final flag.

13.6 Why the last dword is special

The final dword is 00007d64, which is bytes 64 7d 00 00. In ASCII, that is d} followed by two zero bytes. This matches the visible length gate perfectly: the input is still exactly 106 bytes long, while the hidden checker consumes 108 bytes worth of dword slots.

14. Workflow diagram

Figure 7 — Solve workflow diagram.

15. State transition diagram

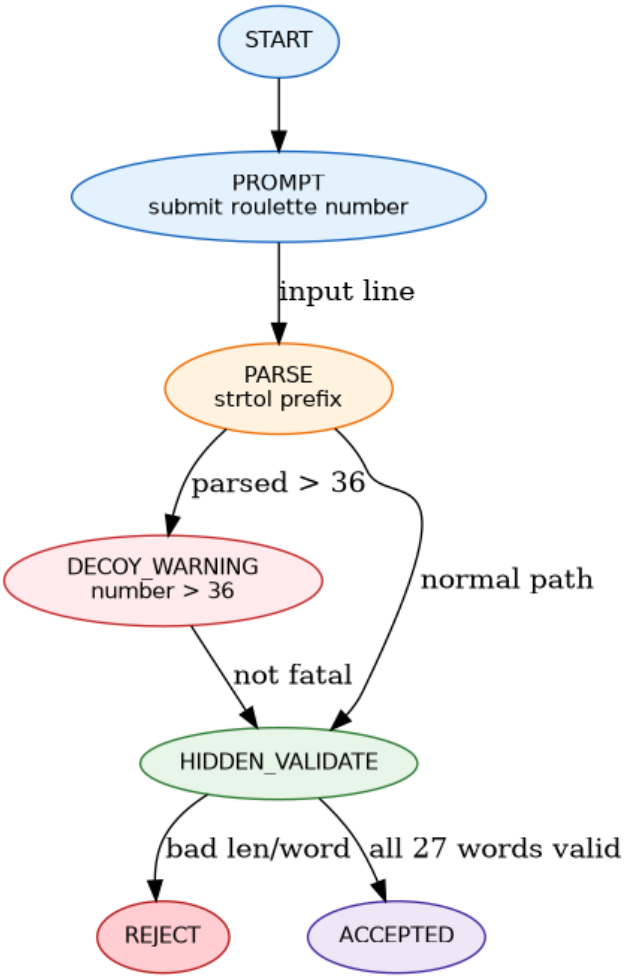


Figure 8 — State transition diagram.

16. High-level pseudocode

```
puts("lets go gambling!");
puts("submit roulette number:");
fgets(line, sizeof(line), stdin);
```

```
strip_newline(line);
parsed = strtol(line, NULL, 10);
if (parsed > 36)
    puts("numbers above 36 aren't valid, dummy");

if (strlen(line) != 0x6a)
    reject();

for (i = 0; i < 27; i++) {
    user_dword = load_le32_with_zero_padding(line + 4*i);
    vm_result  = compute_vm_value(i);
    table_word = *(uint32_t *)(0x499ce0 + 4*i);
    if ((vm_result ^ user_dword) != table_word)
        reject();
}

accept();
```

17. Solver variants and execution commands

These solvers are Python scripts, so there is no compilation stage. The relevant commands are execution commands.

17.1 solve_roulette_bn_static.py - reference solver used in this writeup

This is the reference solver because it reconstructs the accepted input from the 27 recovered little-endian dwords. It matches the reverse-engineering logic presented in the writeup: recovered words -> little-endian packing -> accepted line.

Execution commands:

```
python3 solve_roulette_bn_static.py
python3 solve_roulette_bn_static.py --run ./roulette
python3 solve_roulette_bn_static.py --raw | ./roulette
```

Notes:

- --run ./roulette prints the reconstructed payload and tests it against a local binary.
- --raw writes the raw payload bytes to stdout, which is convenient for piping into the challenge binary.
- This script sends payload + \n in --run mode, which is the cleanest behavior for a program that reads with fgets.

17.2 solve_roulette.py - minimal replay solver

This smaller solver does not reconstruct the payload. It embeds the already recovered final flag string directly and replays it to the binary.

Execution commands:

```
python3 solve_roulette.py
python3 solve_roulette.py --run ./roulette
```

Notes:

- This version is useful for quick local confirmation.
- It is not the primary solver for the writeup because it skips the dword-to-bytes reconstruction step.
- In --run mode it sends the raw flag bytes directly, without appending a final newline.

17.3 Why solve_roulette_bn_static.py is the reference one

solve_roulette_bn_static.py is the better solver for documentation because it shows exactly how the reverse-engineered dwords are turned back into the accepted input. solve_roulette.py is still useful, but it is closer to a replay script than to a reconstruction script.

17.4 Full source of the reference solver

```
#!/usr/bin/env python3
"""
Binary-Ninja-assisted static solver for UMDCTF rev/roulette.

This version does not need ptrace/GDB. It reconstructs the accepted line
from the 27 little-endian dwords recovered after reversing the validator.
"""
import argparse
import struct
import subprocess

WORDS_HEX = """
43444d55 497b4654 4133525f 2d594c4c 746e6177 2d6f742d 79346c70
6568742d 7730702d 61627265 2b2c6c6c 2d747562 642d796d 732d6434
2d646961 732d6f6e 6d692d6f 7433622d 676e6974 6c6c692d 6e69772d
2d6e6f2d 594c4f50 4b52414d 6e695445 61657473 00007d64
"""

def build_payload() -> bytes:
    words = [int(x, 16) for x in WORDS_HEX.split()]
    payload = b"".join(struct.pack("<I", w) for w in words)
    return payload.rstrip(b"\x00")

def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--run", metavar="BIN", help="test payload against a local roulette binary")
    ap.add_argument("--raw", action="store_true", help="print raw payload bytes instead of decoded
text")
    args = ap.parse_args()

    payload = build_payload()
    if args.raw:
        import sys
        sys.stdout.buffer.write(payload + b"\n")
    else:
        print(payload.decode("ascii"))

    if args.run:
        p = subprocess.run([args.run], input=payload + b"\n", stdout=subprocess.PIPE,
stderr=subprocess.STDOUT)
        print(p.stdout.decode(errors="replace"), end="")

if __name__ == "__main__":
    main()
```

18. Verification strategy

- Static confidence: exact length 106 bytes, coherent little-endian decoding, and a flag-shaped plaintext.
- Dynamic confirmation: pipe the reconstructed payload to the local binary and verify that it reaches the accepted path.

19. Main pitfalls and how they were solved

Pitfall	Symptom	Resolution
Looking in the Symbols panel for conditions	You cannot find `if (rax_3 != 0x6a)`	Navigate to `sub_401677` and read the central HLIL/Linear view.
Treating `.rodata` as execution flow	You scroll strings and tables without finding logic	Use xrefs from strings to code.
Believing the roulette UI	You brute-force `0..36`	The numeric parse is a decoy; the real gate is 106 bytes.
Treating the `>36` warning as fatal	You discard long inputs too early	The warning path continues into the real validator.
Confusing rodata objects with the validator array	You over-interpret unrelated tables	Only identify the validator array through xrefs and comparison logic.
Wrong endianness	Dwords decode as garbage	Decode with little-endian packing/unpacking.
Missing final padding	The last dword looks inconsistent	Remember that 106 input bytes are checked as 108 bytes over 27 dwords.

20. Final flag

`UMDCTF{I_R3ALLY-want-to-pl4y-the-p0werball,+but-my-d4d-said-no-so-im-b3tting-ill-win-on-POLYMARKETinstead}`

21. Licence note

Writeup and solver text: CC BY 4.0 / MIT-style educational reuse.

That's All, Folks!